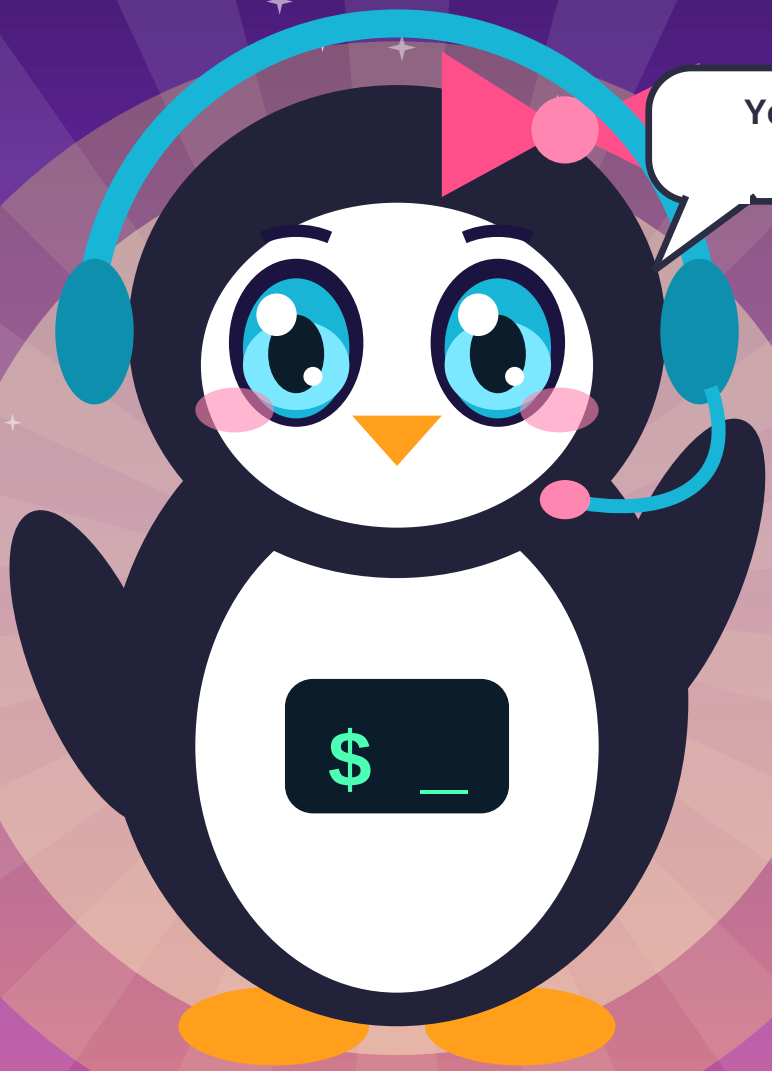
INTERVIEW BOSS BATTLES IV

20 NEW SCENARIOS | LINUX ADMIN | NETWORKING | SECURITY | SCRIPTING



LEGENDARY BOSS: THE LINUX GURU HP 100%

- CREATED BY -
RUTUJA NAIK

SET 4 * 20 LINUX SCENARIOS * 10 RAPID-FIRE

00 HOW TO ANSWER

TRACE for Linux admin scenarios

T — Triage
How bad is it, who is affected, since when, and what changed recently?

R — Read the evidence
Check logs, metrics and state before touching anything.

A — Analyze by layer
Narrow it down: CPU, memory, disk, network, then application and dependencies.

C — Change safely
Smallest fix first, one change at a time, with a rollback ready.

E — Educate and prevent
Monitoring, automation, runbook and a blameless post-mortem.

YOUR 60-90 SECOND ANSWER TEMPLATE

1. Restate the problem and the impact (10 s). **2.** Say your order of hypotheses (10 s). **3.** Walk through 3 to 5 commands and what each tells you (40 s). **4.** State the fix and your rollback plan (15 s). **5.** Finish with prevention: monitoring, automation, documentation (10 s).

POWER PHRASES THAT SOUND SENIOR

"Let me check the logs and the current state first."
"I'd confirm the cause before changing anything."
"I'd test the fix and document it for the team."
"On a production system I'd take a snapshot or backup first."
"I don't know that one offhand, but here is how I would find out."

QUEST LOG (CONTENTS)

01	Heavy swapping and thrashing	2	11	Thousands of TIME_WAIT sockets	12
02	Logs keep growing, logrotate fails	3	12	SSH login takes 30 seconds	13
03	Correct password but cannot log in	4	13	umount says target is busy	14
04	User is not in the sudoers file	5	14	Service keeps restarting	15
05	DNS lookups fail on a Linux server	6	15	Free memory is low: a problem?	16
06	NFS mount hangs the server	7	16	Error loading shared libraries	17
07	apt or yum install fails	8	17	Forgot the root password	18
08	Bash script fails in production	9	18	/boot is full, apt upgrade fails	19
09	SELinux blocks the application	10	19	Roll back a bad package update	20
10	Linux server as a NAT gateway	11	20	Shared team directory setup	21

Troubleshooting | **Admin and security** | **DevOps and cloud** | then: rapid-fire round and final checklist

01 HEAVY SWAPPING AND THRASHING

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

The server is extremely sluggish and vmstat shows constant swap in and out, but nothing has crashed. What do you do?

SAY IT LIKE THIS (60-90 seconds, out loud)

Constant swap in and out is called thrashing. The working set no longer fits in RAM, so the system spends its time moving pages between memory and disk, and everything slows down. I confirm it with `vmstat 1`, looking at the `si` and `so` columns, and `free -m` for swap use. Then I find which processes use the most memory and swap, with `ps` sorted by RSS and the `VmSwap` field in `/proc/PID/status`. I decide whether it's a leak, a genuine spike, or an undersized server. For immediate relief I restart or stop the biggest offender, shed load, or add RAM. For tuning, `vm.swappiness` controls how eagerly the kernel swaps. Lowering it favours RAM for applications, but it doesn't fix a real shortage. Long term: right-size memory, fix leaks, limit workloads with cgroups, and alert on swap activity, not only swap usage.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`vmstat 1 5`	`si` and `so` columns: continuous non-zero values mean active swapping (thrashing).
2	`free -m and swapon --show`	Memory and swap use, and which swap devices exist.
3	`ps -eo pid,rss,vsz,cmd --sort=-rss | head`	Largest processes by resident memory.
4	`grep VmSwap /proc/*/status 2>/dev/null | sort -k2 -n | tail`	Which processes have the most memory swapped out (value in kB).
5	`sysctl vm.swappiness`	How eagerly the kernel swaps (default 60). Lower means prefer RAM.
6	`cat /proc/pressure/memory`	Pressure stall information: how long tasks wait for memory.
7	`sar -B 1 and dmesg -T | grep -i oom`	Paging activity, and whether the OOM killer has started.
8	`sudo sysctl -w vm.swappiness=10`	Temporary tuning only. It eases swapping but does not fix a shortage.

COMMON ROOT CAUSES

Memory leak or growth over time, too many workers or a JVM heap that is too large, an undersized instance, a batch job competing with the app, no memory limits on containers, swappiness too high for the workload.

FIX AND PREVENTION

Reduce memory demand (fix leaks, tune heap and worker counts), add RAM or resize, set cgroup limits (`MemoryMax`), tune swappiness, and alert on swap-in and swap-out rates, which reveal thrashing better than swap usage alone.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Is using some swap always bad?

A. No. Idle pages swapped out can free RAM for cache. Continuous swap-in and swap-out is the problem.

Q. What does swappiness do?

A. A kernel setting (0 to 100, and up to 200 on newer kernels) that controls how aggressively pages move to swap rather than dropping cache.

AVOID THIS MISTAKE

Disabling swap completely as a quick fix. Out-of-memory kills become more likely.

02 LOGS KEEP GROWING, LOGROTATE FAILS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A log file keeps growing to many gigabytes even though logrotate is installed. How do you find out why rotation isn't working?

SAY IT LIKE THIS (60-90 seconds, out loud)

I check three things: is logrotate actually running, does a rule cover this file, and does the rotation succeed. Logrotate normally runs daily from a systemd timer or cron, so I check `systemctl list-timers`. Then I look for a config for this log in `/etc/logrotate.d/`, and I run `logrotate -d` for a dry-run with debug output, and `-f` to force it, to see errors. Common problems are a wrong path or glob, permissions, a syntax error, or the application keeping the old file open after rotation, so it keeps writing to the renamed file. That is solved with a `postrotate` reload or `copytruncate`. I also look at the state file to see when it last rotated, and set size, rotate count, compression and `missingok` sensibly.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`systemctl list-timers | grep logrotate`	Is the daily logrotate job scheduled? (Older systems: /etc/cron.daily/logrotate.)
2	`sudo logrotate -d /etc/logrotate.conf`	Dry run with debug: which files match and what would happen. Shows config errors.
3	`cat /etc/logrotate.d/app`	Is there a rule for this log path? Check the glob and the options.
4	`sudo logrotate -f /etc/logrotate.d/app`	Force a rotation now to test the rule.
5	`cat /var/lib/logrotate/status`	When each log was last rotated. (RHEL: logrotate.status.)
6	`lsof | grep app.log.1`	Is the app still writing to the renamed (rotated) file?
7	`journalctl -u logrotate and grep logrotate /var/log/syslog`	Errors from recent runs: permissions or syntax.
8	Use postrotate or copytruncate	Tell the app to reopen logs, or copy then truncate if it can't.

COMMON ROOT CAUSES

No rule for the file, wrong path or glob, syntax or permission error in the config, the app keeps writing to the old file descriptor after rotation, rotation too infrequent, logrotate timer disabled, or a rule that never triggers because of `size` or `notifempty`.

FIX AND PREVENTION

Add or fix the rule, use `postrotate` to signal the service (or `copytruncate` when needed), set `size`, `rotate`, `compress` and `maxage`, and test with `-d` and `-f`. Alert on large log files and ship important logs to a central system as well.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. copytruncate versus postrotate?

A. copytruncate copies the log and then empties it. It can lose a few lines but needs no app support. postrotate signals the app to reopen its log, which is cleaner.

Q. Why is the disk still full after rotation?

A. If the app still holds the old file open, the space isn't freed until it reopens the log or restarts.

AVOID THIS MISTAKE

Deleting a huge live log with rm. Truncate it or fix rotation so the space is actually freed.

03 CORRECT PASSWORD BUT CANNOT LOG IN

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A user insists their password is correct, but they cannot log in to the server. What do you check?

SAY IT LIKE THIS (60-90 seconds, out loud)

I check the account and authentication layers one at a time. First, does the account exist and have a valid shell? `getent passwd user` shows it, and a shell of `/sbin/nologin` or `/bin/false` blocks interactive logins. Then the password status: `passwd -S` or `chage -l` shows whether the account is locked, the password has expired, or the account itself has expired. Next the logs: `/var/log/auth.log` or `/var/log/secure`, and `journalctl -u sshd`, tell me the exact reason. For SSH I also check `sshd_config` for `AllowUsers`, `AllowGroups` or `DenyUsers`, PAM restrictions such as faillock or access.conf, and whether the user was locked out after repeated failures. If login uses keys, I check permissions on the home directory and `.ssh`.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`getent passwd user`	Does the account exist, and what is its login shell? nologin or false blocks logins.
2	`sudo passwd -S user and sudo chage -l user`	Locked? Password expired? Account expiry date?
3	`sudo faillock --user user`	Is the account temporarily locked after failed attempts? (Older systems: pam_tally2.)
4	`sudo tail -f /var/log/auth.log`	Watch the real reason while the user retries. (RHEL: /var/log/secure.)
5	`sudo sshd -T | grep -iE 'allowusers|allowgroups|denyusers|passwordauth'`	Active sshd rules that may exclude the user.
6	`grep -r user /etc/security/access.conf /etc/ssh/`	PAM or SSH access restrictions.
7	`ls -ld /home/user /home/user/.ssh; ls -l /home/user/.ssh`	Key login needs correct ownership and permissions (700 and 600).
8	`sudo passwd -u user or sudo faillock --user user --reset`	Unlock the account or reset the failure counter, once the cause is understood.

COMMON ROOT CAUSES

Account locked (failed attempts or admin lock), password or account expired, nologin shell, sshd AllowUsers or group restrictions, PAM access rules, wrong key permissions, missing home directory, MFA misconfigured, caps lock or keyboard layout.

FIX AND PREVENTION

Fix the specific cause, then unlock or reset as needed. Keep access in a central directory (LDAP or SSO), use MFA and key-based login, and alert on repeated failures instead of silently locking people out.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does /sbin/nologin do?

A. It is a shell that refuses interactive login and prints a message. It is used for service accounts.

Q. Why might key login fail with a correct key?

A. Strict permissions: the home directory, `.ssh` and `authorized_keys` must not be writable by others, or sshd ignores them (StrictModes).

AVOID THIS MISTAKE

Resetting the password immediately without finding out why login failed.

04 USER IS NOT IN THE SUDOERS FILE

ADMIN & SECURITY DIFFICULTY ✦ ✦ ✦

THE QUESTION

A new team member runs sudo and gets: 'user is not in the sudoers file. This incident will be reported.' How do you fix it properly?

SAY IT LIKE THIS (60-90 seconds, out loud)

That message means the user has no sudo rule. The proper fix is to give access through a group, not by editing the main sudoers file carelessly. On Ubuntu and Debian the `sudo` group has access, on RHEL-family systems it's the `wheel` group. So I add the user to the right group with `usermod -aG`, and the user must log out and in again. For finer control I create a file in `/etc/sudoers.d/` with `visudo -f`, granting only the commands they need, for example restarting one service. I never edit sudoers with a normal editor, because a syntax error can lock everyone out of sudo. I validate with `visudo -c`, test with `sudo -l -U user`, and make sure the grant is recorded in the access process and audited.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`id user and getent group sudo`	Which groups is the user in, and who is in the admin group? (RHEL: wheel.)
2	`sudo usermod -aG sudo user`	Add the user to the admin group (RHEL: wheel). They must log in again.
3	`sudo visudo -f /etc/sudoers.d/deploy`	Create or edit a drop-in file safely. visudo checks syntax before saving.
4	`deploy ALL=(root) NOPASSWD: /usr/bin/systemctl restart app`	Example rule: one user, one command, no full root.
5	`sudo visudo -c`	Validate all sudoers files for syntax errors.
6	`sudo -l -U user`	List exactly what that user may run.
7	`sudo journalctl _COMM=sudo`	Audit trail: who ran which command. (Also /var/log/auth.log.)
8	`ls -l /etc/sudoers.d/`	Drop-in files should be mode 0440 and owned by root.

KEY POINTS TO MENTION

Sudo is rule-based: users or groups are matched to allowed commands. Use groups for normal admin access and drop-in files for limited rights. Least privilege means granting only the commands needed. Everything sudo runs is logged.

FIX AND PREVENTION

Manage sudo access with configuration management (Ansible), use groups from your directory service, avoid NOPASSWD except for narrow, safe commands, and review access regularly.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why use visudo?

A. It locks the file and validates the syntax before saving, so a typo doesn't break sudo for everyone.

Q. What is NOPASSWD and when is it acceptable?

A. It lets a rule run without a password prompt. Acceptable for narrow automation commands, risky for ALL commands.

AVOID THIS MISTAKE

Editing /etc/sudoers with a normal editor. A syntax error can lock out sudo.

05 DNS LOOKUPS FAIL ON A LINUX SERVER

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A Linux server cannot resolve hostnames, though pinging an IP address works. How do you troubleshoot DNS on the host?

SAY IT LIKE THIS (60-90 seconds, out loud)

Since IPs work, the network path is fine and this is name resolution. First I check which resolver the system uses. On modern Ubuntu and many other distributions, `/etc/resolv.conf` points to the local stub `127.0.0.53` of systemd-resolved, so I run `resolvectl status` to see the real upstream DNS servers. Then I test directly with `dig @server name` against each DNS server, to separate a problem in my config from a problem in the DNS server itself. I also check `/etc/nsswitch.conf` for the lookup order, `/etc/hosts` for overrides, search domains, and firewall rules blocking UDP and TCP port 53. In the cloud I verify the VPC DNS settings. Finally I fix the DNS setting at the right layer, such as netplan or NetworkManager, not by hand-editing a generated resolv.conf.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ping -c2 1.1.1.1 and ping -c2 example.com`	Confirms IP connectivity works and only names fail.
2	`cat /etc/resolv.conf and resolvectl status`	Which DNS servers and search domains are really in use?
3	`dig example.com and dig @8.8.8.8 example.com`	Does a known-good public DNS resolve it? Compare with your configured server.
4	`getent hosts example.com`	Resolves the way applications do, using nsswitch (hosts file, DNS and more).
5	`grep hosts /etc/nsswitch.conf and cat /etc/hosts`	Lookup order, and local overrides that may shadow DNS.
6	`nc -zvu DNS_IP 53 and dig +tcp @DNS_IP example.com`	Are UDP and TCP port 53 reachable through the firewall?
7	`sudo resolvectl flush-caches and sudo systemctl restart systemd-resolved`	Clear cached failures and restart the resolver.
8	`nmcli dev show | grep DNS or cat /etc/netplan/*.yaml`	The persistent source of the DNS settings that generate resolv.conf.

COMMON ROOT CAUSES

Wrong or unreachable DNS servers, resolv.conf overwritten by DHCP or NetworkManager, systemd-resolved stub misconfigured, firewall blocking port 53, search domain problems, /etc/hosts overrides, a DNS server outage, DNSSEC or split-horizon issues.

FIX AND PREVENTION

Set DNS in the network manager or netplan, use at least two reachable resolvers, verify firewall rules, and monitor DNS latency. In the cloud, use the provider's VPC resolver or a local caching resolver.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why does resolv.conf show 127.0.0.53?

A. systemd-resolved runs a local stub resolver and forwards to the real upstream servers. Use `resolvectl status` to see them.

Q. dig versus getent hosts?

A. dig queries DNS directly. getent follows the system's full lookup order (hosts file, DNS and so on), like applications do.

AVOID THIS MISTAKE

Hand-editing /etc/resolv.conf. It is usually regenerated and your change disappears.

06 NFS MOUNT HANGS THE SERVER

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Commands like ls and df freeze on a server that has an NFS mount, and processes pile up in uninterruptible state. How do you handle it?

SAY IT LIKE THIS (60-90 seconds, out loud)

That is the classic hung NFS mount. With the default `hard` mounts, processes that touch the share wait forever, in D state, for the NFS server to respond, which is why even `df` hangs. First I confirm the cause: check connectivity to the NFS server, use `showmount` or `rpcinfo`, and read `dmesg` for 'nfs: server not responding'. If the server or network comes back, the processes recover. If not, I try a forced lazy unmount, `umount -f -l`, to detach it, and as a last resort I restart the clients. Long term, mount options matter: `soft` returns errors instead of hanging but risks data corruption, while `hard` is safer for data. Timeouts, `_netdev` and `x-systemd.automount` help. I'd also fix the root cause: server load, network or firewall.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`dmesg -T | grep -i nfs`	'nfs: server X not responding, still trying' confirms the hang.
2	`mount | grep nfs and nfsstat -m`	Which NFS mounts exist, with their options (hard or soft, timeouts, version).
3	`ping NFS_SERVER and rpcinfo -p NFS_SERVER`	Is the server reachable, and are its RPC services up?
4	`showmount -e NFS_SERVER`	Are the exports still available?
5	`ps -eo pid,stat,wchan:20,cmd | awk '$2 ~ /D/'`	Processes stuck in uninterruptible sleep, and what they wait on.
6	`timeout 5 ls /mnt/share`	Safe test: don't let your own shell freeze on a hung mount.
7	`sudo umount -f -l /mnt/share`	Forced, lazy unmount: detaches immediately, cleans up when it is no longer busy.
8	fstab: hard,timeo=600,retrans=2,_netdev,x-systemd.automount	Sensible options: wait for network, mount on demand, bounded retries.

COMMON ROOT CAUSES

NFS server down or overloaded, network outage or a firewall change, DNS failure for the server name, stale file handle after the export changed, mismatched NFS versions, `hard` mounts with no timeout, kernel or client bugs.

FIX AND PREVENTION

Restore the server or network, lazy-unmount if needed, and prefer automount with sensible timeouts. Monitor NFS latency and server health, avoid NFS for critical paths, and consider alternatives (object storage, replicated storage) where appropriate.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Hard versus soft mounts?

A. Hard retries forever (safe for data, but can hang). Soft gives up after retries and returns an error (can cause silent data corruption).

Q. What is a stale file handle?

A. The client refers to a file or export that no longer exists on the server, for example after the share was re-created. Remounting fixes it.

AVOID THIS MISTAKE

Running df or ls over and over on the hung mount. Each one gets stuck too.

07 APT OR YUM INSTALL FAILS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

apt-get install fails with 'Could not get lock /var/lib/dpkg/lock-frontend' or 'broken packages'. How do you fix it safely?

SAY IT LIKE THIS (60-90 seconds, out loud)

A lock error means another package process is running, such as an unattended upgrade, so I first check with `ps` and wait for it to finish, rather than deleting lock files. I remove a stale lock only after confirming no process uses it. For broken packages, I run `dpkg --configure -a` to finish interrupted installs, then `apt-get -f install` to fix dependencies, and `apt update` to refresh. If `apt update` itself fails, I read the error: a 404 means a dead repository or wrong release, 'GPG error: NO_PUBKEY' means a missing signing key, and timeouts point to network or proxy problems. On RHEL-family systems the same ideas apply with `dnf clean all`, `dnf makecache` and `dnf history`. I always check what a command will change before confirming.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ps aux | grep -E 'apt|dpkg|unattended'`	Is another package process running? Wait for it.
2	`sudo lsof /var/lib/dpkg/lock-frontend`	Which process holds the lock?
3	`sudo dpkg --configure -a`	Finish packages that were left half-configured.
4	`sudo apt-get -f install`	Fix broken dependencies.
5	`sudo apt update (read the errors)`	404 = dead repo or wrong release. NO_PUBKEY = missing key. Timeout = network or proxy.
6	`apt-cache policy package and cat /etc/apt/sources.list.d/*`	Which versions and repositories are in use?
7	`sudo dnf clean all && sudo dnf makecache`	RHEL family: reset and rebuild the metadata cache.
8	`sudo apt-get install -s package`	Simulate an install to see what would change before running it.

COMMON ROOT CAUSES

Another package process running (unattended-upgrades), an interrupted installation, dependency conflicts, dead or wrong repositories, missing GPG keys, no disk space (especially /boot or /var), mixed releases or PPAs, proxy problems.

FIX AND PREVENTION

Wait for or stop the other process, repair dpkg and dependencies, correct the repository configuration, and avoid mixing repositories. Test upgrades in staging and use configuration management to keep repository setups consistent.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why not just delete the lock file?

A. If a process is really running, removing the lock can corrupt the package database. Only remove it after confirming nothing uses it.

Q. What does apt-get -f install do?

A. It tries to repair broken dependencies by installing or removing the packages needed to reach a consistent state.

AVOID THIS MISTAKE

Deleting lock files or forcing installs without checking what is running.

08 BASH SCRIPT FAILS IN PRODUCTION

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A bash script that worked on your laptop fails in production, sometimes silently. How do you debug it and make it more robust?

SAY IT LIKE THIS (60-90 seconds, out loud)

First I reproduce it with tracing: `bash -x script.sh` prints each command as it runs, or I wrap the suspect section in `set -x`. Then I look at the usual differences between environments: PATH, the shell (sh versus bash), tool versions (GNU versus BSD), permissions, line endings and environment variables. Silent failures usually come from unchecked exit codes, unquoted variables that break on spaces, or pipelines where only the last command's status counts. So I harden the script with `set -euo pipefail`, quote variables, validate inputs, use `trap` for cleanup and error reporting, use full paths or check that required commands exist, and log with timestamps. I lint it with ShellCheck, which catches many of these bugs, and I run it in a clean container to mimic production.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`bash -x script.sh (or set -x ... set +x)`	Trace every command with its expanded arguments.
2	`bash -n script.sh and shellcheck script.sh`	Syntax check, and lint for quoting, globbing and portability problems.
3	`head -1 script.sh and file script.sh`	Check the shebang. CRLF line endings (from Windows) cause 'bad interpreter'.
4	`set -euo pipefail`	Fail fast on errors, unset variables and failed pipeline stages.
5	`echo "$?" and ${PIPESTATUS[@]}`	Exit status of the last command, and of every stage of a pipeline.
6	`trap 'echo "failed at line $LINENO" >&2' ERR`	Report where it failed.
7	`env -i bash script.sh`	Run with an empty environment to expose hidden dependencies on variables and PATH.
8	`docker run --rm -v $PWD:/s ubuntu:24.04 bash /s/script.sh`	Test in a clean environment similar to production.

COMMON ROOT CAUSES

Unquoted variables with spaces, unchecked exit codes, different PATH or tool versions, sh versus bash differences, CRLF line endings, relying on the current directory, missing permissions, locale differences, environment variables set only in your own shell.

FIX AND PREVENTION

Add strict mode, quote everything, use absolute paths or `cd` to the script's directory, validate inputs, log errors, lint with ShellCheck in CI, and consider a more structured language when scripts grow large.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Single versus double quotes in bash?

A. Double quotes expand variables and command substitution. Single quotes keep everything literal.

Q. Why quote variables?

A. Without quotes, values with spaces or wildcards are split and expanded, causing bugs and even dangerous commands like rm.

AVOID THIS MISTAKE

Sprinkling `|| true` everywhere. It hides the real failure.

09 SELINUX BLOCKS THE APPLICATION

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

Your application works on one server but fails on a RHEL server with no obvious error. You suspect SELinux. How do you confirm it and fix it without disabling SELinux?

SAY IT LIKE THIS (60-90 seconds, out loud)

SELinux is mandatory access control on top of normal permissions, so something can be denied even if file permissions look perfect. I confirm with `getenforce`, then look for AVC denials with `ausearch -m avc -ts recent`. To prove it's SELinux, I can switch to permissive temporarily with `setenforce 0`. If the problem disappears, SELinux was the cause, and I switch back right away. Then I fix it the right way: wrong file labels are corrected with `restorecon` or `semanage fcontext`, non-standard ports are allowed with `semanage port`, and behaviour is enabled with booleans like `httpd_can_network_connect`. If no rule fits, I generate a small local policy module with `audit2allow` after reviewing exactly what it permits. I never leave SELinux disabled.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`getenforce and sestatus`	Enforcing, permissive or disabled?
2	`sudo ausearch -m avc -ts recent`	Recent denials: source process, target file or port, and the denied permission.
3	`sudo sealert -a /var/log/audit/audit.log`	(setroubleshoot) Human-readable explanations and suggested fixes.
4	`ls -Z /path/file and ps -eZ | grep app`	File and process security contexts.
5	`sudo setenforce 0 (test only), then setenforce 1`	Temporary permissive mode to confirm SELinux is the cause.
6	`sudo restorecon -Rv /var/www/html`	Restore default labels on files that have the wrong context.
7	`sudo semanage fcontext -a -t httpd_sys_content_t '/data/web(/.*)?'`	Define labels for a custom path, then run restorecon.
8	`sudo semanage port -a -t http_port_t -p tcp 8081`	Allow a non-standard port. For behaviour, use `setsebool -P name 1`.

COMMON ROOT CAUSES

Files copied or moved with the wrong label (mv keeps the old context), non-standard ports or directories, a missing boolean for network or database connections, a service running in a confined domain nobody expected.

FIX AND PREVENTION

Fix labels, ports or booleans, and use `audit2allow` only for reviewed, minimal rules. Keep SELinux enforcing, include contexts in your automation, and test new deployments in enforcing mode.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why does mv cause problems but cp doesn't?

A. mv keeps the file's original SELinux context. cp creates a new file that inherits the destination directory's context.

Q. What is a boolean in SELinux?

A. A switch that turns predefined policy behaviours on or off, such as allowing a web server to connect to the network.

AVOID THIS MISTAKE

Running setenforce 0, or disabling SELinux permanently, to make the error go away.

10 LINUX SERVER AS A NAT GATEWAY

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

How do you turn a Linux server into a NAT gateway, so machines in a private network can reach the internet through it?

SAY IT LIKE THIS (60-90 seconds, out loud)

I'd do three things: enable IP forwarding in the kernel, add a NAT masquerade rule on the outgoing interface, and allow forwarded traffic in the firewall. Forwarding is `net.ipv4.ip_forward=1`, set persistently in `/etc/sysctl.d`. Then with nftables or iptables I masquerade traffic from the private subnet leaving through the public interface, so replies come back to the gateway. I also need forward rules that allow established and related traffic back in, and new traffic from the private network out, and deny the rest. The private hosts use the gateway as their default route, with working DNS. I test with `ping`, `curl` and `tcpdump` on both interfaces, and make the rules persistent. In the cloud I'd normally use a managed NAT gateway, since a self-managed NAT instance is a single point of failure.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`sysctl net.ipv4.ip_forward`	Is forwarding on? 1 means yes.
2	`echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-forward.conf && sudo sysctl --system`	Enable it now and keep it after reboot.
3	`sudo iptables -t nat -A POSTROUTING -s 10.0.1.0/24 -o eth0 -j MASQUERADE`	Source-NAT private traffic that leaves through eth0.
4	`sudo iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT`	Allow the private side to start outbound connections.
5	`sudo iptables -A FORWARD -i eth0 -o eth1 -m state --state RELATED,ESTABLISHED -j ACCEPT`	Allow only return traffic back in.
6	`sudo iptables -t nat -L -n -v and sudo nft list ruleset`	Verify the rules and watch the packet counters increase.
7	`sudo tcpdump -nn -i eth1 icmp and sudo tcpdump -nn -i eth0 icmp`	Follow a test ping across both interfaces.
8	`sudo apt install iptables-persistent`	Make the rules survive reboots (or use the nftables service).

KEY POINTS TO MENTION

NAT lets many private addresses share one public address. Masquerade rewrites the source address and tracks connections. Forwarding must be enabled and the FORWARD chain must allow the traffic. Clients need the gateway as default route and a DNS server.

FIX AND PREVENTION

Use nftables or firewalld for maintainable rules, restrict forwarding to the needed subnets, log and monitor, and in the cloud prefer managed NAT gateways or a highly available NAT with failover.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. SNAT versus masquerade?

A. Both rewrite the source address. SNAT uses a fixed address. Masquerade uses the outgoing interface's current address, which suits dynamic IPs.

Q. What is DNAT used for?

A. Rewriting the destination, for example port forwarding from a public port to a private server.

AVOID THIS MISTAKE

Forgetting the persistent sysctl setting. Forwarding stops working after a reboot.

11 THOUSANDS OF TIME_WAIT SOCKETS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A busy server shows tens of thousands of connections in TIME_WAIT, and clients sometimes fail with 'Cannot assign requested address'. What is happening?

SAY IT LIKE THIS (60-90 seconds, out loud)

TIME_WAIT is a normal TCP state: the side that closes a connection first keeps the socket for about 60 seconds, to make sure late packets are handled. A few thousand is normal. But if a client or proxy opens a huge number of short connections to the same destination, it can run out of ephemeral ports, which is the range the kernel uses for outgoing connections, giving 'Cannot assign requested address'. I'd first count states with `ss -s`, check the ephemeral port range, and identify which side closes the connections and why there are so many short-lived ones. The best fix is in the application: use keep-alive and connection pooling to reuse connections. System tuning can help: widen `ip_local_port_range`, spread across more destination IPs, and use `tcp_tw_reuse` for outgoing connections. I avoid the old `tcp_tw_recycle`, which was removed because it breaks clients behind NAT.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ss -s`	Totals for TCP states, including how many are timewait.
2	`ss -tan state time-wait | awk '{print $4}' | cut -d: -f1 | sort | uniq -c | sort -rn | head`	Which remote addresses hold the most TIME_WAIT sockets (IPv4 view).
3	`sysctl net.ipv4.ip_local_port_range`	The ephemeral port range (default about 32768 to 60999, roughly 28,000 ports).
4	`sudo sysctl -w net.ipv4.ip_local_port_range='15000 65000'`	Widen the range for outgoing connections.
5	`sysctl net.ipv4.tcp_tw_reuse`	Allows safe reuse of TIME_WAIT sockets for new outgoing connections.
6	`sysctl net.ipv4.tcp_fin_timeout`	A related timer. It does not shorten TIME_WAIT, which is fixed at 60 seconds.
7	Application: keep-alive and connection pooling	Reuse connections instead of opening a new one per request.
8	`ss -tan | awk '{print $1}' | sort | uniq -c`	Compare ESTABLISHED, TIME-WAIT and CLOSE-WAIT counts.

COMMON ROOT CAUSES

Clients or reverse proxies opening a new connection per request, no keep-alive or pooling, a load test from a single source IP, many short health checks, a small ephemeral port range, many connections to a single destination IP and port.

FIX AND PREVENTION

Enable HTTP keep-alive and pooling, spread connections over more destination IPs or ports, widen the port range, enable `tcp_tw_reuse` for clients, and monitor socket states. Don't follow old advice about `tcp_tw_recycle`.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Who ends up in TIME_WAIT?

A. The side that closes the connection first. On a busy proxy or client, that is often the machine running out of ports.

Q. What is the ephemeral port range?

A. The local ports the kernel assigns to outgoing connections. Each combination of source IP, destination IP and destination port can use each local port once.

AVOID THIS MISTAKE

Copying the old tcp_tw_recycle advice from the internet.

12 SSH LOGIN TAKES 30 SECONDS

TROUBLESHOOTING DIFFICULTY ✦ ✦ ✦

THE QUESTION

SSH connections to a server connect quickly but then take 20 to 30 seconds before the prompt appears. How do you find and fix the delay?

SAY IT LIKE THIS (60-90 seconds, out loud)

Where the delay occurs tells me the cause. With `ssh -vvv` I watch the timing to see which step stalls. The classic cause is reverse DNS: sshd tries to resolve the client's IP address (`UseDNS`) and waits for a DNS timeout. Another is GSSAPI or Kerberos authentication attempts timing out. A third, if it stalls after authentication, is PAM or systemd-logind, for example slow LDAP lookups, or a slow `.bashrc` or login script. I'd test with `-o GSSAPIAuthentication=no`, check the server's sshd config for `UseDNS` and `GSSAPIAuthentication`, check the DNS resolver settings, and read the auth log timestamps. Then I change the specific setting and reload sshd safely.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ssh -vvv user@host`	Debug output. See which step pauses: DNS, GSSAPI, key exchange or authentication.
2	`ssh -o GSSAPIAuthentication=no user@host`	Quick test: if it is fast now, GSSAPI was the delay.
3	`sudo sshd -T | grep -iE 'usedns|gssapi'`	Server settings: UseDNS and GSSAPIAuthentication.
4	`sudo tail -f /var/log/auth.log`	Timestamps for each stage of the login.
5	`time getent hosts CLIENT_IP and resolvectl status`	Is reverse DNS slow or failing? Check the resolver.
6	`journalctl -u systemd-logind`	Slow session creation after authentication.
7	`time bash -lc exit`	Time the login scripts (.bash_profile, .bashrc, profile.d).
8	`sudo sshd -t && sudo systemctl reload sshd`	Validate and apply the config change safely.

COMMON ROOT CAUSES

Reverse DNS lookups timing out (UseDNS), GSSAPI or Kerberos attempts, slow or unreachable LDAP or SSSD during PAM, systemd-logind delays, heavy login scripts, wrong resolver settings.

FIX AND PREVENTION

Set `UseDNS no` and `GSSAPIAuthentication no` if you don't need them, repair DNS or LDAP reachability, trim login scripts, and measure again. Keep the change in configuration management.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does UseDNS do?

A. It makes sshd look up (and verify) the client's hostname, which can hang if reverse DNS is slow. Modern versions default to no.

Q. How do you debug the client side?

A. Use `ssh -vvv` and compare the timing between lines to see where it pauses.

AVOID THIS MISTAKE

Restarting sshd over your only SSH session without validating the config first.

13 UMOUNT SAYS TARGET IS BUSY

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

umount /data fails with 'target is busy'. How do you find what is using it and unmount safely?

SAY IT LIKE THIS (60-90 seconds, out loud)

Something has files open on that mount, or a process's current directory is inside it. I find it with `lsof +f -- /data` or `fuser -vm /data`, which list the processes and users. Common holders are an application, a shell sitting in that directory, a backup job, a Samba or NFS export, or even a container bind mount. I stop the application cleanly or ask the owner to close it, change my own shell's directory, and unmount again. Only if I have to, I use `fuser -k` to kill the processes, which is destructive, or a lazy unmount, `umount -l`, which detaches the mount from the tree and cleans up once it's no longer busy. For a disk removal or filesystem check, a lazy unmount isn't enough: nothing must be using it.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`fuser -vm /data`	Lists processes (PID, user, access type) using the mount.
2	`sudo lsof +f -- /data`	Open files on that filesystem, with the command and path.
3	`pwd and cd /`	Is your own shell inside the mount? Move out.
4	`systemctl stop app`	Stop the service that uses the mount, cleanly.
5	`findmnt -R /data`	Are there nested mounts or bind mounts below it? Unmount those first.
6	`docker ps -q | xargs -I{} docker inspect -f '{{.Name}} {{.Mounts}}' {}`	Do containers bind-mount that path?
7	`sudo fuser -km /data`	Kill every process using the mount. Destructive: last resort.
8	`sudo umount -l /data`	Lazy unmount: detaches now, finishes when it is no longer busy.

COMMON ROOT CAUSES

An application or service with open files, a shell whose current directory is inside, nested or bind mounts, containers using the path, NFS or Samba exports, a swap file or loop device on it, a backup or indexer running.

FIX AND PREVENTION

Stop services cleanly, close sessions, unmount nested mounts first, and use lazy unmount only when appropriate. For maintenance, plan the order: stop apps, unmount, then check or detach the disk.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does umount -l do?

A. Lazy unmount: removes the mount from the filesystem tree immediately and cleans up after processes stop using it. The data may still be in use.

Q. Why does my own shell block the unmount?

A. If your current directory is inside the mount, the filesystem is in use. Change directory out first.

AVOID THIS MISTAKE

Using fuser -k on a production mount without knowing what it will kill.

14 SERVICE KEEPS RESTARTING

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A systemd service keeps restarting and then fails with 'Start request repeated too quickly'. What does it mean and how do you fix it?

SAY IT LIKE THIS (60-90 seconds, out loud)

systemd has a rate limit on restarts. If a service with `Restart=always` or `on-failure` crashes repeatedly within a short window, defined by `StartLimitIntervalSec` and `StartLimitBurst`, systemd stops trying and marks it failed with that message. So the message is a symptom, and the real problem is why the service crashes. I read `systemctl status` and `journalctl -u service -b` to see the exit code or signal and the application's last log lines. Typical causes are bad config, a missing dependency, a port conflict, permission errors, a missing environment file, or the service starting before its dependencies are ready. After fixing the cause I clear the failed state with `systemctl reset-failed`. I can also tune `RestartSec` to add a delay between attempts, so it doesn't spin.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`systemctl status app -l`	State, exit code or signal and the last log lines. Look for `status=1/FAILURE` or `SEGV`.
2	`journalctl -u app -b --no-pager | tail -50`	The application's error output before each crash.
3	`systemctl cat app`	Check Restart=, RestartSec=, StartLimitBurst and ExecStart.
4	`sudo -u appuser /usr/bin/app --config /etc/app.conf`	Run the command by hand as the service user to see the real error.
5	`ss -tulpn | grep :PORT and ls -l /etc/app.conf`	Port conflict? Permissions or missing files?
6	`systemd-analyze verify /etc/systemd/system/app.service`	Check the unit file for mistakes.
7	`sudo systemctl reset-failed app && sudo systemctl start app`	Clear the rate-limit state after fixing the cause.
8	`RestartSec=5 StartLimitIntervalSec=60 StartLimitBurst=5`	Add delay between restarts and set the allowed number of attempts.

COMMON ROOT CAUSES

Bad config or a missing file, port already in use, permission or SELinux problems, missing environment variables or dependencies, an application crash or exit, starting before dependencies are ready, a wrong `Type=` setting (for example forking versus simple).

FIX AND PREVENTION

Fix the underlying error, set `RestartSec` and sensible limits, order the unit after its dependencies, and use `Type=` correctly. Add health monitoring and alerts for services entering the failed state.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Restart=always versus on-failure?

A. always restarts however the service exits. on-failure restarts only on a non-zero exit, a signal or a timeout, not on a clean exit.

Q. What does reset-failed do?

A. It clears the failed state and the restart counters, so you can start the service again.

AVOID THIS MISTAKE

Raising or disabling the start limit instead of fixing the crash.

15 FREE MEMORY IS LOW: A PROBLEM?

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Monitoring shows only 200 MB of free memory on a server with 16 GB RAM. Is that a problem, and how do you find out?

SAY IT LIKE THIS (60-90 seconds, out loud)

Not necessarily. Linux uses spare RAM for the page cache and buffers to speed up disk access, and gives it back when applications need it, so low 'free' memory is normal. What matters is the `available` column in `free -m`, which estimates memory that can be used without swapping. I check available memory, swap activity and the memory pressure indicators. If available is healthy and there's no swapping, there is no problem. If available is low and swap is active, I look for the real consumers: `ps` sorted by RSS, `smem` for proportional set size, and `/proc/meminfo` for slab and shared memory. I also check cgroup or container limits. I'd configure alerts on available memory and swap-in rate, not on free memory.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`free -m`	Read `available`, not `free`. Cache and buffers are reclaimable.
2	`grep -E 'MemAvailable|Cached|Buffers|Slab|Shmem|SwapTotal|SwapFree' /proc/meminfo`	Detailed breakdown, including kernel slab and shared memory.
3	`vmstat 1 5`	Is there swap in and out (si, so)? If not, there is no pressure.
4	`cat /proc/pressure/memory`	Memory pressure stall information: real contention.
5	`ps -eo pid,rss,cmd --sort=-rss | head`	Biggest processes by resident memory.
6	`smem -rtk (if installed)`	PSS-based per-process memory, which accounts for shared pages.
7	`sudo slabtop -o | head`	Kernel slab usage (dentries, inodes), if the kernel seems to hold a lot of memory.
8	`systemd-cgtop and docker stats`	Per-service and per-container memory compared with their limits.

KEY POINTS TO MENTION

Used memory includes cache that the kernel frees when needed. `available` estimates what applications can still get. There is a problem only when available is low, swapping is active, pressure is high, or OOM kills occur.

FIX AND PREVENTION

Alert on `MemAvailable` and swap activity. Investigate only real pressure: leaks, oversized caches, wrong limits. Don't drop caches in production just to make the numbers look better.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What is the page cache?

A. Memory the kernel uses to cache file data, which makes repeated reads fast. It is released automatically under memory pressure.

Q. What is the difference between RSS and PSS?

A. RSS counts all resident pages a process uses, including shared ones. PSS divides shared pages among the processes using them, so totals add up correctly.

AVOID THIS MISTAKE

Regularly running echo 3 > /proc/sys/vm/drop_caches. It hurts performance and fixes nothing.

16 ERROR LOADING SHARED LIBRARIES

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Running a program fails with 'error while loading shared libraries: libxyz.so.1: cannot open shared object file'. How do you fix it?

SAY IT LIKE THIS (60-90 seconds, out loud)

The dynamic linker can't find a library the program needs. I use `ldd` on the binary to list its dependencies and see which are 'not found'. Then I decide: is the library simply not installed, or is it installed in a location the linker doesn't search? If it's missing, I find the package that provides it, for example with `apt-file search` or `dnf provides`, and install it. If it exists in a non-standard path, I add that directory to `/etc/ld.so.conf.d/`, run `ldconfig`, or use `LD_LIBRARY_PATH` for a temporary test. I also check for 32-bit versus 64-bit mismatches and wrong versions, such as `libxyz.so.1` versus `.so.2`. For software I installed myself, an `rpath` or proper packaging avoids the problem.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ldd /path/to/program`	Lists needed libraries. Lines with `=> not found` are the missing ones.
2	`ldconfig -p | grep libxyz`	Is the library known to the dynamic linker cache?
3	`find / -name 'libxyz.so*' 2>/dev/null`	Does the library exist anywhere on the system?
4	`apt-file search libxyz.so.1 (or dnf provides '*/libxyz.so.1')`	Which package provides it? Install that package.
5	`echo '/opt/app/lib' | sudo tee /etc/ld.so.conf.d/app.conf && sudo ldconfig`	Add a custom library directory permanently and rebuild the cache.
6	`LD_LIBRARY_PATH=/opt/app/lib ./program`	Temporary test only. Avoid relying on it in production.
7	`file /path/to/program and uname -m`	Check the architecture (32-bit versus 64-bit, x86_64 versus arm64).
8	`readelf -d /path/to/program | grep -E 'NEEDED|RPATH|RUNPATH'`	Libraries and paths the binary was linked with.

COMMON ROOT CAUSES

Library package not installed, library in a non-standard directory, ldconfig cache not updated, version mismatch (so.1 versus so.2), wrong architecture, a binary built on a newer distribution than the target, a broken symlink.

FIX AND PREVENTION

Install the right package, register custom paths with ld.so.conf.d and ldconfig, build or package the software properly (static linking, rpath, or a container), and keep builds and targets on the same distribution family.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does ldconfig do?

A. It rebuilds the cache of shared libraries (`/etc/ld.so.cache`) from the configured directories, so the linker can find them quickly.

Q. Why avoid LD_LIBRARY_PATH in production?

A. It affects every program started with it, and can load the wrong libraries or create security risks. Prefer system configuration or rpath.

AVOID THIS MISTAKE

Copying libraries around by hand or symlinking mismatched versions.

17 FORGOT THE ROOT PASSWORD

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

You are locked out of a Linux server because nobody knows the root password. How do you regain access?

SAY IT LIKE THIS (60-90 seconds, out loud)

With physical or console access, I can reset it by interrupting the boot. On RHEL-family systems I edit the GRUB entry and add `rd.break`, which drops me into an emergency shell in the initramfs. There I remount `/sysroot` read-write, `chroot /sysroot`, run `passwd root`, and `touch /.autorelabel` so SELinux relabels files on the next boot, then exit and reboot. On Ubuntu and Debian I can use recovery mode, or add `init=/bin/bash` to the kernel line, remount root read-write, and set the password. In the cloud I'd instead use the provider's serial console or SSM, or attach the disk to another instance, and I'd check whether a sudo-capable user already exists. I'd also mention the security angle: without a GRUB password or disk encryption, physical access means root.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	**Reboot, press e at GRUB, add rd.break to the linux line, press Ctrl+X**	RHEL family: stops in the initramfs before the real root is mounted.
2	`mount -o remount,rw /sysroot`	Make the root filesystem writable.
3	`chroot /sysroot`	Switch into the real system.
4	`passwd root`	Set the new root password.
5	`touch /.autorelabel`	Make SELinux relabel files on next boot so the change works.
6	`exit; exit (then reboot)`	Leave the chroot and the shell, and boot normally.
7	**Ubuntu or Debian: recovery mode, or init=/bin/bash then mount -o remount,rw /**	Alternative route to a root shell, then run passwd.
8	**Cloud: serial console, SSM, or attach the disk to a rescue instance**	No GRUB access over the network: edit /etc/shadow or sudoers from the rescue instance.

KEY POINTS TO MENTION

Anyone with console access and no disk encryption or GRUB password can become root, which is why physical security and encryption matter. SELinux needs a relabel, or login may fail after the change.

FIX AND PREVENTION

Prefer named admin accounts with sudo and SSH keys, so the root password is rarely needed. Protect GRUB with a password, use full-disk encryption, store break-glass credentials in a vault, and audit console access.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does rd.break do?

A. It stops the boot in the initramfs before switching to the real root filesystem, giving you a root shell for repairs.

Q. Why touch /.autorelabel?

A. The password file you changed now has a wrong SELinux context. The flag makes the system relabel everything at the next boot.

AVOID THIS MISTAKE

Skipping the SELinux relabel step on RHEL, and then being unable to log in.

18 /BOOT IS FULL, APT UPGRADE FAILS

TROUBLESHOOTING DIFFICULTY ✦ ✦ ✦

THE QUESTION

apt upgrade fails because /boot has no space left. How do you clean it up safely?

SAY IT LIKE THIS (60-90 seconds, out loud)

/boot is usually a small partition, and it fills up with old kernels and initramfs images after many updates. I check with `df -h /boot` and list the installed kernels. The rule is to keep the running kernel plus at least one older known-good kernel, and remove the rest. First I confirm which kernel is running with `uname -r`. Then I remove old ones with `apt autoremove --purge`, which removes unused kernels automatically, or remove specific versions explicitly. On RHEL, `dnf remove --oldinstallonly` or limiting `installonly_limit` does the same. If apt is already broken because of the full disk, I free a little space first, then run `dpkg --configure -a` and `apt -f install`. Long term I enable automatic removal of unused kernels and size /boot properly.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`df -h /boot and ls -lh /boot`	How full is it, and which files take the space?
2	`uname -r`	Never remove the running kernel.
3	`dpkg -l 'linux-image-*' | grep ^ii`	List installed kernel packages. (RHEL: rpm -q kernel.)
4	`sudo apt autoremove --purge`	Remove unused kernels and dependencies automatically.
5	`sudo apt remove --purge linux-image-5.15.0-91-generic`	Remove a specific old kernel (keep the running one plus one older).
6	`sudo dnf remove --oldinstallonly --setopt installonly_limit=2 kernel`	RHEL family: remove old kernels, keeping two.
7	`sudo dpkg --configure -a and sudo apt -f install`	Finish the interrupted upgrade after space is freed.
8	`sudo update-grub`	Regenerate the GRUB config after removing kernels.

COMMON ROOT CAUSES

Many old kernels accumulated, a small /boot partition, unattended upgrades not removing old kernels, large initramfs files, other files stored in /boot.

FIX AND PREVENTION

Enable `Unattended-Upgrade::Remove-Unused-Kernel-Packages`, set `installonly_limit` on RHEL, size /boot at 1 GB or more on new systems, and monitor /boot usage.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why is /boot often a separate partition?

A. Historically because of bootloader limits, and in encrypted or LVM setups where the bootloader must read it before the rest is unlocked.

Q. What lives in /boot?

A. The kernel (vmlinuz), initramfs images, GRUB files and configuration, and System.map.

AVOID THIS MISTAKE

Deleting files in /boot with rm instead of removing packages properly.

19 ROLL BACK A BAD PACKAGE UPDATE

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A package update broke an application. How do you roll it back safely, and how do you stop it from updating again?

SAY IT LIKE THIS (60-90 seconds, out loud)

First I confirm which package and version changed, from the package manager history: `/var/log/apt/history.log` on Debian-based systems or `dnf history` on RHEL. Then I install the previous version: with apt, `apt install package=version`, using `apt-cache policy` to see the available versions, and with dnf, `dnf downgrade package` or `dnf history undo`. If the old version is no longer in the repository, I take it from the local cache in `/var/cache/apt/archives`, a snapshot, or an internal mirror. After the rollback, I hold the package so updates don't reinstall it, with `apt-mark hold` or `dnf versionlock`. I test the app, then raise the issue with the vendor. For prevention: test updates in staging, use snapshots before patching, and run a local mirror with pinned snapshots.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`grep -B2 -A5 package /var/log/apt/history.log`	What was upgraded, when, and from which version? (RHEL: dnf history.)
2	`apt-cache policy package`	Installed version, candidate version and available versions.
3	`sudo apt install package=1.2.3-1`	Install the previous version (apt asks to confirm the downgrade).
4	`sudo dnf downgrade package or sudo dnf history undo ID`	RHEL family: roll back a package or a whole transaction.
5	`ls /var/cache/apt/archives/ | grep package`	Is the old .deb still cached? Install it with dpkg -i.
6	`sudo apt-mark hold package`	Prevent the package from being upgraded again. (RHEL: dnf versionlock add.)
7	`systemctl restart app`	Restart the application and run smoke tests.
8	Snapshot or image rollback	If the VM has a snapshot from before patching, restoring it is often fastest.

COMMON ROOT CAUSES

A new version with a bug, a changed default config, a dependency or library upgrade, ABI or API changes, incompatible plugins or modules, or config files replaced during the upgrade.

FIX AND PREVENTION

Test updates in staging first, snapshot before patching, use a local repository mirror with fixed snapshots, pin critical packages, and keep a documented rollback procedure. Report the bug upstream.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does apt-mark hold do?

A. It marks a package as held back, so apt won't upgrade, install or remove it until you unhold it.

Q. Why can't you always downgrade?

A. The old version may no longer be in the repository, or dependencies and data formats may have changed. Snapshots are safer.

AVOID THIS MISTAKE

Leaving a package on hold forever. Held packages miss security fixes, so review them.

20 SHARED TEAM DIRECTORY SETUP

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A team needs a shared directory on a server: group members can read and write, others get no access, and new files should keep the group. How do you set this up?

SAY IT LIKE THIS (60-90 seconds, out loud)

I'd use a group, the right permissions, the setgid bit and a sensible umask. First I create a group, for example `devs`, and add users with `usermod -aG`. Then I create the directory, set its group with `chgrp devs`, and set mode `2770`: the leading 2 is setgid, so new files and folders inherit the group of the directory, and 770 gives owner and group full access and others none. The umask matters too. With a umask of 022, new files wouldn't be group-writable, so I use a default ACL or a group umask to make them group-writable. For finer control, such as one extra user with read-only access, I use ACLs with `setfacl`. Users must log in again for new group membership, and I verify with a test user.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`sudo groupadd devs && sudo usermod -aG devs alice`	Create the group and add users. They must log in again.
2	`sudo mkdir /srv/shared && sudo chgrp devs /srv/shared`	Create the directory and set its group.
3	`sudo chmod 2770 /srv/shared`	setgid (2) plus rwx for owner and group, nothing for others.
4	`ls -ld /srv/shared`	Check for `drwxrws---`. The `s` in the group position is setgid.
5	`sudo setfacl -d -m g:devs:rwx /srv/shared`	Default ACL: new files inherit group rwx regardless of umask.
6	`sudo setfacl -m u:auditor:rx /srv/shared`	Give one extra user read-only access without changing the group.
7	`getfacl /srv/shared`	Verify ACLs and defaults.
8	`sudo -u alice touch /srv/shared/test && ls -l /srv/shared`	Test as a real user. The file should belong to group devs.

KEY POINTS TO MENTION

setgid on a directory makes new files inherit the directory's group. Umask decides default permissions, so group-write may be missing without ACLs or a group umask. ACLs add per-user or per-group rules. New group membership needs a new login session.

FIX AND PREVENTION

Use groups instead of individual permissions, document them, manage them with configuration management or your directory service, and review access regularly. Avoid 777.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does the s mean in drwxrws---?

A. The setgid bit: new files and folders inherit the directory's group.

Q. Why isn't group write working for new files?

A. The creating user's umask (often 022) removes group write. Use a default ACL or a umask of 002 for that group.

AVOID THIS MISTAKE

Using chmod 777 on the shared directory.

21 RAPID-FIRE ROUND

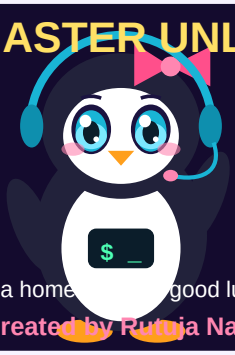
10 quick questions, 1 to 2 sentence answers

1	**Hard link versus symbolic link?**	A hard link is another name for the same inode, so it survives deletion of the original name. A symbolic link is a separate file that points to a path and breaks if the target moves.
2	**What are the fields in /etc/passwd?**	Seven colon-separated fields: username, x (password lives in shadow), UID, GID, comment, home directory and login shell.
3	**cron versus at?**	cron runs jobs repeatedly on a schedule. at runs a job once at a given time.
4	**What does umask do?**	It removes permissions from newly created files. With umask 022, new files are 644 and directories 755.
5	**Runlevels versus systemd targets?**	Runlevels were the old SysV numbers 0 to 6. systemd targets replace them, such as multi-user.target (3) and graphical.target (5).
6	**apt versus dpkg?**	dpkg installs a single .deb and doesn't fetch or resolve dependencies. apt downloads from repositories and resolves dependencies, using dpkg underneath.
7	**kill versus killall versus pkill?**	kill signals a PID. killall signals processes by exact name. pkill matches by pattern, and with -f it matches the full command line.
8	**Login versus non-login shell?**	Login shells read /etc/profile and ~/.bash_profile. Interactive non-login shells read ~/.bashrc. Many setups source .bashrc from .bash_profile.
9	**/dev/null, /dev/zero, /dev/urandom?**	/dev/null discards everything written to it. /dev/zero gives endless zero bytes. /dev/urandom gives random bytes.
10	**nohup versus & ?**	& runs a job in the background, but it may die when the shell exits (SIGHUP). nohup makes the command ignore SIGHUP and writes output to nohup.out.

BEFORE THE INTERVIEW: FINAL CHECKLIST

- Practise each answer out loud in 60 to 90 seconds. Record yourself once.
- Prepare a real example for each topic: "Last time I saw this, I..."
- Know why a command works, not just its syntax.
- Admit what you don't know, then explain how you would find out.
- End answers with prevention: monitoring, automation, documentation.
- Keep a home lab (VM, Docker, minikube). Break things on purpose and fix them.

LINUX MASTER UNLOCKED!



Practice out loud, build a home [illegible] good luck with your interviews.

Created by Rutuja Naik